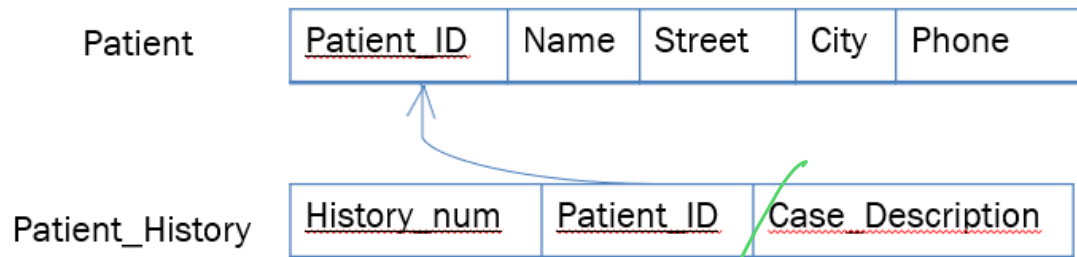


Create, insert, update, delete tables



-- Creating table patient

```
CREATE TABLE patient (  
  patient_id INTEGER PRIMARY KEY,  
  name VARCHAR(20),  
  street VARCHAR(20),  
  city VARCHAR(20),  
  phone VARCHAR(10));
```

```
CREATE TABLE patient (  
  patient_id INTEGER,  
  name VARCHAR(20),  
  street VARCHAR(20),  
  city VARCHAR(20),  
  phone VARCHAR(10),  
  PRIMARY KEY(patient_id));
```

-- DESCRIBE the table

```
DESCRIBE patient;
```

-- Delete table

```
DROP TABLE patient;
```

-- Constraints

```
CREATE TABLE patient (  
  patient_id INTEGER PRIMARY KEY,  
  name VARCHAR(20) NOT NULL,  
  street VARCHAR(20),  
  city VARCHAR(20) DEFAULT 'Cairo',  
  phone VARCHAR(10) UNIQUE);
```

-- Creating table patient_history (with foreign key)

```
CREATE TABLE patient_history (  
  history_num INTEGER,  
  patient_id INTEGER,  
  case_description VARCHAR(20),  
  FOREIGN KEY(patient_id) REFERENCES patient (patient_id),  
  PRIMARY KEY(history_num, patient_id));
```

-- Add column to table

```
ALTER TABLE patient  
ADD country VARCHAR(20);
```

-- Rename column

```
ALTER TABLE patient  
RENAME COLUMN country TO patient_country;
```

-- Delete column from table

```
ALTER TABLE patient  
DROP COLUMN patient_country;
```

-- Rename table

```
ALTER TABLE patient RENAME TO patients;
```

-- Modifying column constraint

```
ALTER TABLE patient_history  
MODIFY case_description VARCHAR(25) NOT NULL;
```

-- Insert data

```
INSERT INTO patient VALUES(1, 'mohamed', 'mohamed ali', 'cairo', '1111');  
INSERT INTO patient VALUES(2, 'ahmed', 'hassanen hiekal', 'mansoura', '2222');  
INSERT INTO patient (patient_id, name) VALUES(3, 'ali');  
INSERT INTO patient (patient_id, name, phone) VALUES(4, 'mahmoud', '5555');  
INSERT INTO patient VALUES(5, 'hussien', 'ramses', 'alex', '3333');
```

-- Update data within table

```
UPDATE patient  
SET name = 'Johnny'  
WHERE patient_id = 4;
```

```
UPDATE patient  
SET phone = '4444'  
WHERE name = 'ali';
```

```
UPDATE patient  
SET phone = '6666'  
WHERE name = 'ahmed' AND patient_id = 2;
```

```
UPDATE patient
SET street = 'shoubra'
WHERE street IS NULL;
```

```
UPDATE patient
SET phone = '7777', name = 'mohamed'
WHERE patient_id = 2;
```

-- Delete data from table

```
DELETE FROM patient
WHERE patient_id = 4;
```

```
DELETE FROM patient
WHERE phone = '3333' AND name = 'hussien';
```

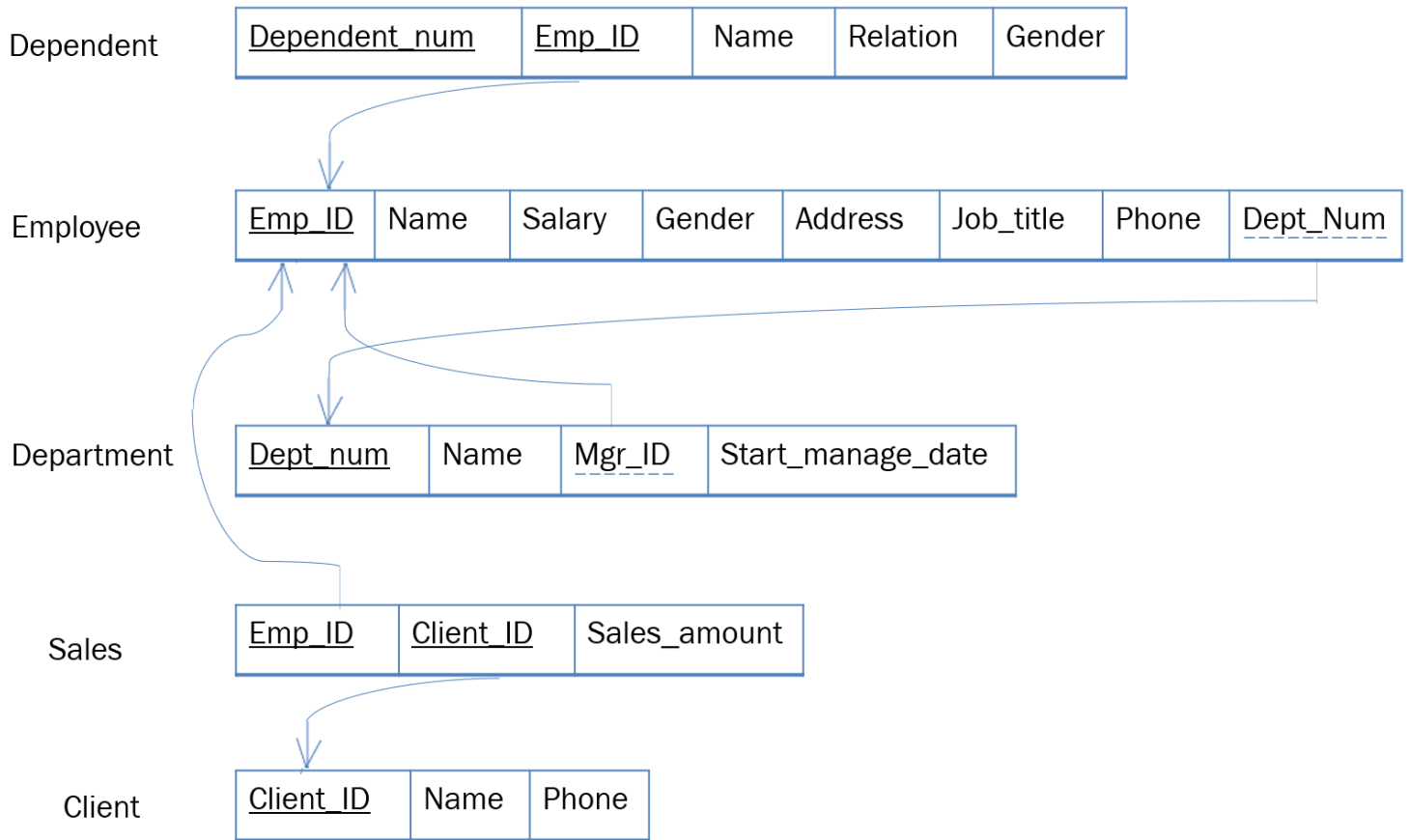
-- Delete all data from table

```
DELETE FROM patient;
```

-- Delete related tables

```
DROP TABLE patient_history;
DROP TABLE patient;
```

Creating Company Database:



```
CREATE TABLE employee (  
  emp_id INTEGER PRIMARY KEY,  
  name VARCHAR(40),  
  salary INTEGER,  
  gender VARCHAR(1),  
  address VARCHAR(40),  
  job_title VARCHAR(40),  
  phone VARCHAR(10),  
  dept_num INTEGER);  
-- FOREIGN KEY(dept_num) REFERENCES department(dept_num)
```

```
CREATE TABLE department (  
  dept_num INTEGER PRIMARY KEY,  
  name VARCHAR(20),  
  mgr_id INTEGER,  
  mgr_start_date DATE,  
  FOREIGN KEY(mgr_id) REFERENCES employee(emp_id));
```

```
ALTER TABLE employee  
ADD FOREIGN KEY(dept_num) REFERENCES department(dept_num);
```

```
CREATE TABLE dependent (  
  dependent_num INTEGER,  
  emp_id INTEGER,  
  name VARCHAR(20),  
  relation VARCHAR(20),  
  gender VARCHAR(1),  
  FOREIGN KEY(emp_id) REFERENCES employee(emp_id),  
  PRIMARY KEY(dependent_num, emp_id));
```

```
CREATE TABLE client (  
  client_id INTEGER PRIMARY KEY,  
  name VARCHAR(40),  
  phone VARCHAR(10));
```

```
CREATE TABLE sales (  
  emp_id INTEGER,  
  client_id INTEGER,  
  sales_amount DECIMAL(10,2),
```

```
PRIMARY KEY(emp_id, client_id),  
FOREIGN KEY(emp_id) REFERENCES employee(emp_id),  
FOREIGN KEY(client_id) REFERENCES client(client_id));
```

-- Dropping tables that have relations between each other

```
DROP TABLE sales;
```

```
DROP TABLE client;
```

```
DROP TABLE dependent;
```

```
ALTER TABLE employee  
DROP COLUMN dept_num;
```

```
DROP table employee;
```

```
DROP table department;
```

Queries:

-- Find all employees

```
SELECT *  
FROM employees;
```

-- Find the first and last names of all employees

```
SELECT first_name, last_name  
FROM employees;
```

-- Find the forename and surnames names of all employees (Alias)

```
SELECT first_name, last_name AS family_name  
FROM employees;
```

-- Find out all the different jobs in the company

```
SELECT job_id  
FROM jobs;
```

-- Find out all the different jobs in the company, that already has hired employees

```
SELECT DISTINCT job_id  
FROM employees;
```

-- Find all the employees who work as 'SH_CLERK'

```
SELECT *  
FROM employees  
WHERE job_id= 'SH_CLERK';
```

-- Find all employees at department 100

```
SELECT *
```



```
FROM employees
WHERE department_id = 100;
```

-- Find all employees ordered by salary

```
SELECT *
from employees
ORDER BY salary ASC;          -- ASC: is the default
```

-- Find all employees ordered by their hire-date from older to newer

```
SELECT *
FROM   employees
ORDER BY hire_date;          -- ASC: is the default
```

-- Find all employees ordered by their job_id then name

```
SELECT *
from employees
ORDER BY job_id, first_name;
```

-- Find all employees ordered by job_id in ascending order then by salary in descending order.

```
SELECT *
from employees
ORDER BY job_id ASC, salary DESC;
```

-- Find the first and last names of the employees who work in department 100, ordered by their family name in ascending order.

```
SELECT first_name, last_name
FROM employees
```

WHERE department_id = 100

ORDER BY last_name;

-- Find all employee's id, names, and their annual salary, ordered by annual salary

SELECT employee_id, first_name, last_name, salary*12 ann_sal

FROM employees

ORDER BY ann_sal;

-- Find all employee's id's and names who were hired beginning from 2002

SELECT employee_id, first_name, last_name

FROM employees

WHERE hire_date >= '01-JAN-2002';

-- The emp_id and names of employees, who work as 'SH_CLERK' and their salaries are less than 3000

SELECT employee_id, first_name, salary

FROM employees

WHERE job_id = 'SH_CLERK' AND salary < 3000;

-- Find all employees who are 'SH_CLERK' & hired after 2007, or who make over 10000

SELECT *

FROM employees

WHERE (hire_date > '30-DEC-2007' AND job_id = 'SH_CLERK') OR salary > 10000;

-- Find all employees are hired between 2001 and 2005

SELECT *

FROM employees

WHERE hire_date BETWEEN '01-JAN-2001' AND '30-DEC-2005';

-- WHERE hire_date >= '01-JAN-2001' AND hire_date <= '30-DEC-2005';

-- Find all employees are not hired between 2001 and 2005

SELECT *

FROM employees

WHERE hire_date NOT BETWEEN '01-JAN-2001' AND '30-DEC-2005';

-- Find the employee named 'Steven'

SELECT *

FROM employees

WHERE LOWER(first_name) = 'steven';

-- INITCAP(first_name) = 'Steven';

-- UPPER(first_name) = 'STEVEN';

-- Find all employees named Steven, Den, or John

SELECT *

FROM employees

WHERE first_name IN ('Steven', 'Den', 'John');

-- WHERE first_name = 'Steven' OR first_name = 'Den' OR first_name = 'John';

-- Find all employees not named Steven, Den, or John

SELECT *

FROM employees

WHERE first_name NOT IN ('Steven', 'Den', 'John');

```
-- WHERE first_name != 'Steven' AND first_name != 'Den' AND first_name != 'John';
```

```
-- Find all employees that have no manager
```

```
SELECT *
```

```
FROM employees
```

```
WHERE manager_id IS NULL;
```

```
-- NULL cannot be compared using =
```

```
-- Find all employees that have managers
```

```
SELECT *
```

```
FROM employees
```

```
WHERE manager_id IS NOT NULL;
```

Wildcards

-- % = any number of characters, _ = one character

-- Find all employees with salary is higher than 3000 and less than 4000

SELECT *

FROM employees

WHERE salary >= 3000 AND salary < 4000

-- WHERE salary BETWEEN 3000 AND 3999;

-- WHERE salary LIKE '3____';

-- Find any job that titled as manager

SELECT *

FROM jobs

WHERE job_title LIKE '%Manager';

-- Find any employee hired on October

-- '15-OCT-99'

-- '15-OCT-1999'

-- '15-OCTOBER-99'

-- '15-OCTOBER-1999'

SELECT *

FROM employees

WHERE LOWER(hire_date) LIKE ('__-oct%');

-- Find any employee hired on the 10th day of the month

SELECT *

FROM employees

WHERE hire_date LIKE '10%';

-- Find the employee_id and name of employees that their names start with any letter in range: A - F

SELECT employee_id, first_name

From employees

WHERE first_name BETWEEN 'A%' AND 'F%';

-- WHERE LOWER(first_name) IN ('a%', 'b%', 'c%', 'd%', 'e%', 'f%');

Aggregation functions

-- Find the number of employees

```
SELECT COUNT(employee_id)
FROM employees;
```

```
SELECT COUNT(*)
FROM employees;
```

-- Find the number of employees that have managers

```
SELECT COUNT(manager_id)
FROM employees;
```

-- Find the average of all employee's salaries

```
SELECT AVG(salary)
FROM employees;
```

-- Find the sum of all employee's salaries

```
SELECT SUM(salary)
FROM employees;
```

-- Find the number of employees who work as 'SH_CLERK' and hired after 2005

```
SELECT COUNT(*)
FROM employees
WHERE job_id = 'SH_CLERK' AND hire_date > '30-DEC-2005';
```

-- Find out how many employees in each job_id

```
SELECT job_id, COUNT(*)
FROM employees
```

```
GROUP BY job_id;
```

-- GROUP BY is used only with the aggregation functions

-- functions are calculated for each group separately

-- Find the maximum salary of employee in each department

```
SELECT department_id, MAX(salary)
```

```
FROM employees
```

```
GROUP BY department_id;
```

-- Find the sum salaries in each department, ordered in a descending order.

```
SELECT department_id, SUM(salary)
```

```
FROM employees
```

```
GROUP BY department_id
```

```
ORDER BY SUM(salary) DESC;
```

-- Find the sum salaries in each department, displaying only the values higher than 50000.

```
SELECT department_id, SUM(salary)
```

```
FROM employees
```

```
GROUP BY department_id
```

```
HAVING SUM(salary) > 50000;
```

-- HAVING is used only with GROUP BY

-- WHERE is used to filter out the individual rows, while HAVING is used to filter out the groups

-- Find the sum salaries for department 100 only

```
SELECT department_id, SUM(salary)
```

```
FROM employees
```



```
GROUP BY department_id  
HAVING department_id = 100;
```

```
SELECT SUM (salary)  
FROM employees  
WHERE department_id = 100;
```

```
-- Find how many job_id in the company  
-- finding job_id that already has employees
```

```
SELECT COUNT (DISTINCT job_id)  
FROM employees;
```

```
-- finding all job_id in the company, whether it has employees or not
```

```
SELECT COUNT (job_id)  
FROM jobs;
```